

What Black *can't* do is decide where blank lines within your functions, methods, or global scope should go. Which of those lines to group together is a subjective decision that is up to the programmer.

For example, let's look at the `EmailValidator` class in `validators.py` in the Django web app framework. It's not necessary for you to understand how this code works. But pay attention to how blank lines separate the `__call__()` method's code into four groups:

```
--snip--
def __call__(self, value):
    ❶ if not value or '@' not in value:
        raise ValidationError(self.message, code=self.code)

    ❷ user_part, domain_part = value.rsplit('@', 1)

    ❸ if not self.user_regex.match(user_part):
        raise ValidationError(self.message, code=self.code)

    ❹ if (domain_part not in self.domain_whitelist and
        not self.validate_domain_part(domain_part)):
        # Try for possible IDN domain-part
        try:
            domain_part = punycode(domain_part)
        except UnicodeError:
            pass
        else:
            if self.validate_domain_part(domain_part):
                return
            raise ValidationError(self.message, code=self.code)
--snip--
```

Even though there are no comments to describe this part of the code, the blank lines indicate that the groups are conceptually distinct from each other. The first group ❶ checks for an `@` symbol in the `value` parameter. This task is different from that of the second group ❷, which splits the email address string in `value` into two new variables, `user_part` and `domain_part`. The third ❸ and fourth ❹ groups use these variables to validate the user and domain parts of the email address, respectively.

Although the fourth group has 11 lines, far more than the other groups, they're all related to the same task of validating the domain of the email